

LOAN APPROVAL

Prepared by: Rahul Sharma
rahsharmaz465@gmail.com



INTRODUCTION

In this assessment, you will perform an Exploratory Data Analysis (EDA) on a dataset related to home loan approval. The dataset is sourced from Skill Circle and contains valuable information for analysis. The primary focus of this assessment is on data exploration and visualization machine learning techniques are not required.

THE GOALS OF THIS ASSESSMENT ARE TO:

Gain familiarity with the dataset. Perform data exploration and visualization. Identify patterns, trends, and potential insights.

Generate meaningful visualizations to communicate your findings.

PROJECT TASKS

Task 1: Data Exploration

Load the dataset into a Python environment (e.g., Jupyter Notebook).

Display the first few rows of the dataset to understand its structure.

Check for missing values and handle them if necessary.

Summarize basic statistics (mean, median, standard deviation, etc.) for the numeric columns.

Task 2: Data Visualization

2.1 Univariate Analysis

Explore the distribution of numeric columns using the following visualizations:

Histograms: Plot the frequency distribution of key numeric variables.

Box Plots: Identify potential outliers and visualize the spread of data.

Analyze categorical variables by creating the following plots:

Bar Charts: Visualize the frequency distribution of categorical variables.

Pie Charts: Represent the composition of categorical variables.

2.2 Bivariate Analysis

Create scatter plots to explore relationships between pairs of numeric variables.

Use pair plots (scatter matrix) to visualize interactions between multiple numeric variables simultaneously.

Investigate the relationship between categorical and numeric variables using box plots or violin plots.

2.3 Multivariate Analysis

Perform a correlation analysis to identify relationships between numeric variables.

Visualize correlations using a heatmap.

Create a stacked bar chart to show the distribution of categorical variables across multiple categories.

Task 3: Geospatial Analysis (Optional)

If the dataset contains geographical information, visualize data on a map to identify regional trends.

Use scatter plots or heatmaps to display data patterns across geographic locations.

DATA EXPLORATION

Load the dataset

```
from google.colab import drive
drive.mount('/content/drive')
```

```
import pandas as pd
import numpy as np
df=pd.read_csv("/content/drive/MyDrive/loan_sanction_test.csv")
```

df

...	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
0	LP001015	Male	Yes	0.000000	Graduate	No	5720
1	LP001022	Male	Yes	1.000000	Graduate	No	3076
2	LP001031	Male	Yes	2.000000	Graduate	No	5000
3	LP001035	Male	Yes	2.000000	Graduate	No	2340
4	LP001051	Male	No	0.000000	Not Graduate	No	3276
...	...	...	...	...	...	...	...
362	LP002971	Male	Yes	0.555205	Not Graduate	Yes	4009
363	LP002975	Male	Yes	0.000000	Graduate	No	4158
364	LP002980	Male	No	0.000000	Graduate	No	3250
365	LP002986	Male	Yes	0.000000	Graduate	No	5000
366	LP002989	Male	No	0.000000	Graduate	Yes	9200

367 rows × 12 columns

Display the first few rows of the dataset to understand its structure.

 `df.head()`

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome
0	LP001015	Male	Yes	0	Graduate	No	5720	0
1	LP001022	Male	Yes	1	Graduate	No	3076	1500
2	LP001031	Male	Yes	2	Graduate	No	5000	1800
3	LP001035	Male	Yes	2	Graduate	No	2340	2546
4	LP001051	Male	No	0	Not Graduate	No	3276	0

Check for missing values and handle them if necessary.

 `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Loan_ID               367 non-null   object
 1   Gender                356 non-null   object
 2   Married               367 non-null   object
 3   Dependents            357 non-null   object
 4   Education             367 non-null   object
 5   Self_Employed         344 non-null   object
 6   ApplicantIncome       367 non-null   int64
 7   CoapplicantIncome     367 non-null   int64
 8   LoanAmount            362 non-null   float64
 9   Loan_Amount_Term      361 non-null   float64
10  Credit_History        338 non-null   float64
11  Property_Area         367 non-null   object
dtypes: float64(3), int64(2), object(7)
memory usage: 34.5+ KB
```

```
df.isnull().sum()
```

	0
Loan_ID	0
Gender	11
Married	0
Dependents	10
Education	0
Self_Employed	23
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	5
Loan_Amount_Term	6
Credit_History	29
Property_Area	0

Summarize basic statistics (mean, median, standard deviation, etc.) for the numeric columns.

```
df["Gender"].mode()
```

Gender

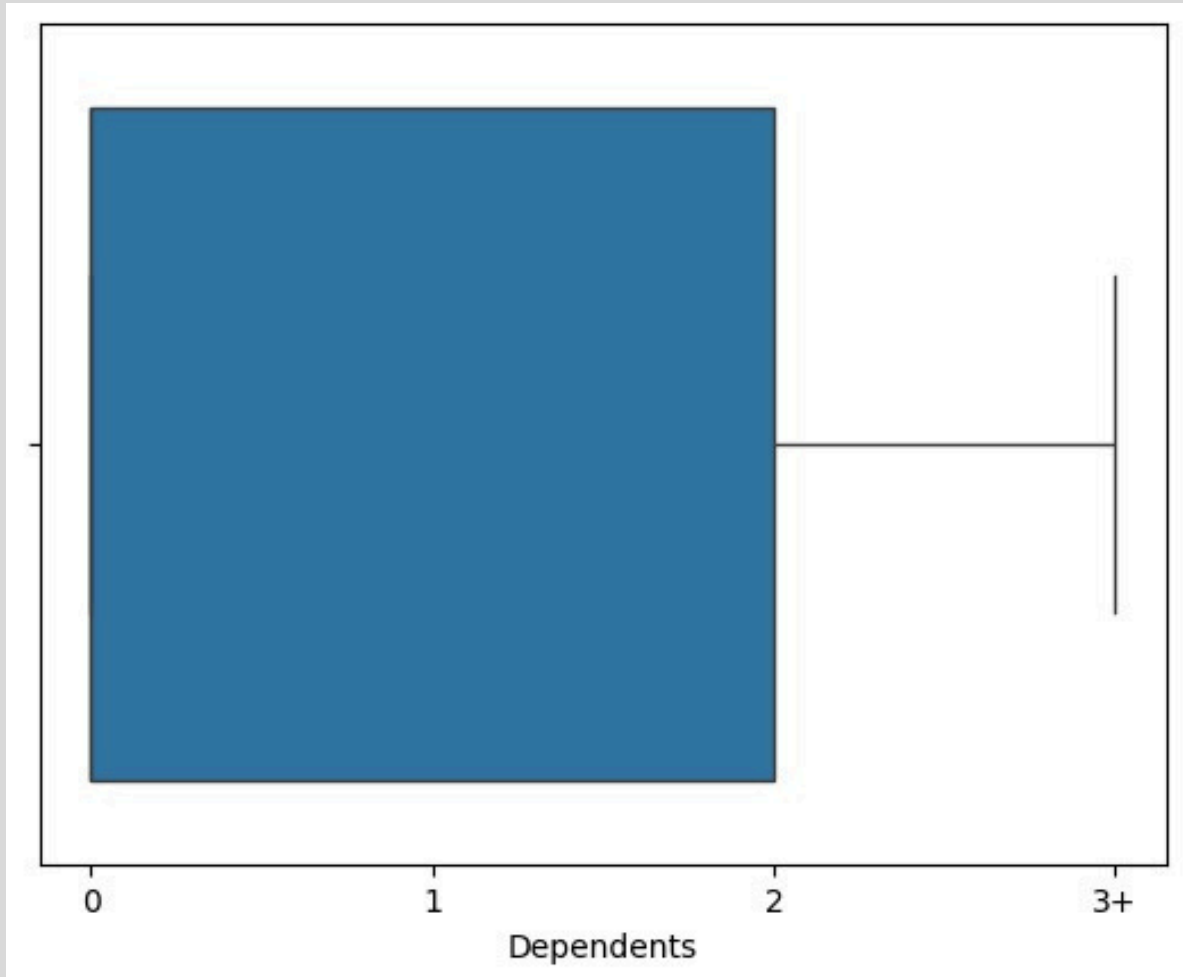
0 Male

dtype: object

```
df["Gender"].fillna("Male", inplace=True)
```

MAKE BOXPLOT TO CHECK HOW MANY OUTLIERS ARE THEIR

```
import seaborn as sns
sns.boxplot(data=df,x="Dependents")
```



To check in Dependents Column there are any Symbol
if Yes it will be replaced with NaN (missing value)
instead

```
print(df['Dependents'].unique())
df['Dependents']=pd.to_numeric(df['Dependents'], errors='coerce')
```


BECAUSE OUTLIER IS ABSENT

```
mean_dependents=df["Dependents"].mean()  
print(f"The mean of the 'Dependents' column is: {mean_dependents}")
```

```
... The mean of the 'Dependents' column is: 0.555205047318612
```

```
df["Dependents"].fillna(0.555205047318612, inplace=True)
```

in Object Type use Mode (Aggregate Function) to find
(Frequently Occuring Terms)

```
df["Self_Employed"].mode()
```

```
...      Self_Employed  
0           No
```

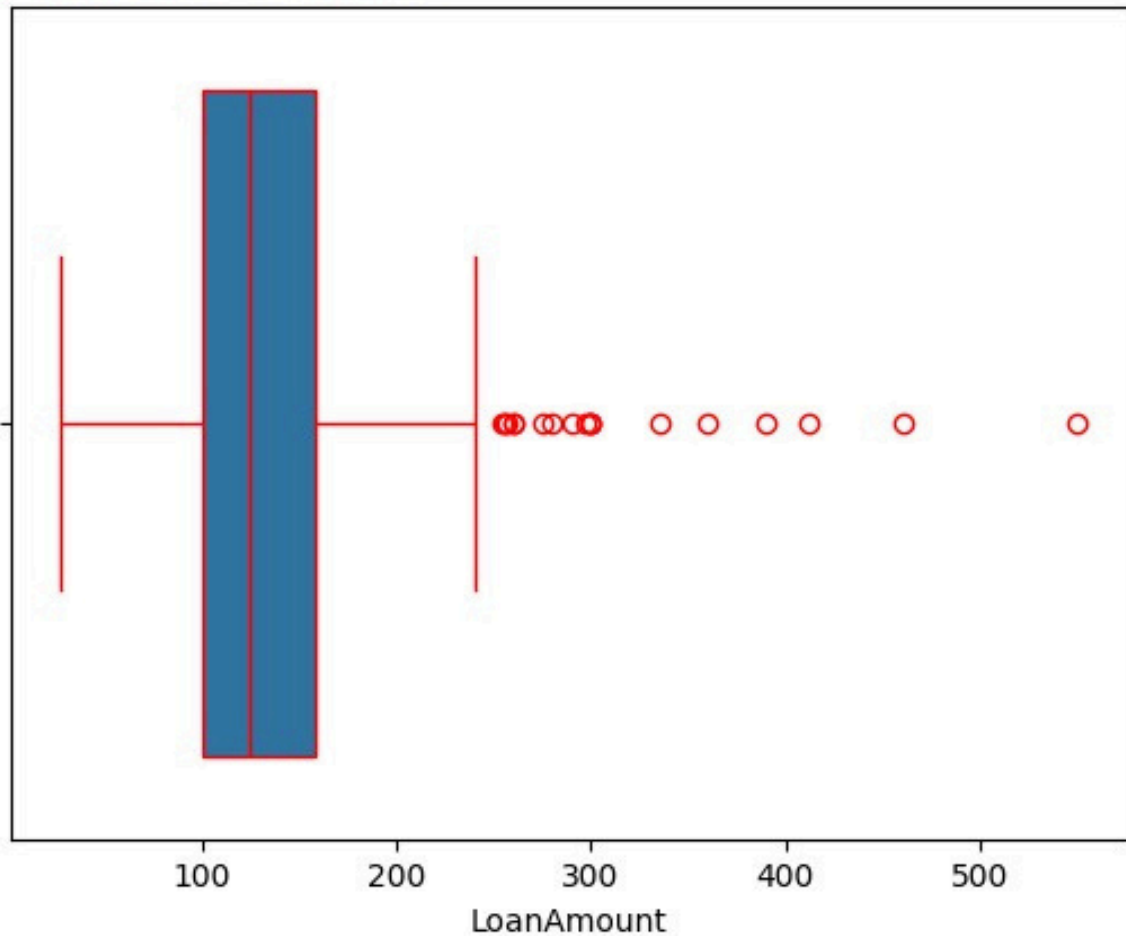
```
dtype: object
```

```
df["Self_Employed"].fillna("No", inplace=True)
```

MAKE BOXPLOT TO CHECK HOW MANY OUTLIERS ARE THEIR

```
import seaborn as sns
sns.boxplot(x="LoanAmount", data=df, linecolor="Red")
```

<Axes: xlabel='LoanAmount'>



There are Outliers in Loan Amount columns

```

import numpy as np
col=df['LoanAmount']
Q1=col.quantile(0.25)
Q3=col.quantile(0.75)

IQR = Q3 - Q1

lower_fence=Q1-(1.5 * IQR)
upper_fence=Q3 +(1.5 * IQR)

print("Lower Limit:",lower_fence)
print("Upper Limit:",upper_fence)

# Find outliers
outliers= df[(col < lower_fence) | (col > upper_fence)] # | is 'Or'
print("Outliers:\n",outliers)

```

```

Lower Limit: 13.625
Upper Limit: 244.625
Outliers:

```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
8	LP001059	Male	Yes	2.000000	Graduate	No	
18	LP001108	Male	Yes	0.000000	Graduate	No	
24	LP001149	Male	Yes	0.000000	Graduate	No	
27	LP001169	Male	Yes	0.000000	Graduate	No	
81	LP001428	Male	Yes	0.555205	Graduate	No	
83	LP001446	Male	Yes	0.000000	Graduate	No	
91	LP001483	Male	Yes	0.555205	Graduate	No	
96	LP001500	Male	Yes	1.000000	Graduate	No	
124	LP001655	Female	No	0.000000	Graduate	No	
143	LP001791	Male	Yes	0.000000	Graduate	Yes	
144	LP001794	Male	Yes	2.000000	Graduate	Yes	
189	LP002059	Male	Yes	2.000000	Graduate	No	
194	LP002077	Male	Yes	1.000000	Graduate	No	
284	LP002570	Female	Yes	2.000000	Graduate	No	
285	LP002572	Male	Yes	1.000000	Graduate	No	
331	LP002825	Male	Yes	0.555205	Graduate	No	
345	LP002878	Male	Yes	0.555205	Graduate	No	
350	LP002899	Male	Yes	2.000000	Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
8	13633	0	280.0	240.0	
18	9226	7916	300.0	360.0	
24	5400	4380	290.0	360.0	
27	7500	3750	275.0	360.0	
81	72529	0	360.0	360.0	
83	8449	0	257.0	360.0	
91	13518	0	390.0	360.0	
96	3333	4200	256.0	360.0	
124	12500	0	300.0	360.0	
143	32000	0	550.0	360.0	
144	10890	0	260.0	12.0	
189	7874	3967	336.0	360.0	
194	10000	2690	412.0	360.0	
284	10000	11666	460.0	360.0	
285	8750	0	297.0	360.0	
331	9699	0	300.0	360.0	
345	8334	0	260.0	360.0	
350	8667	0	254.0	360.0	

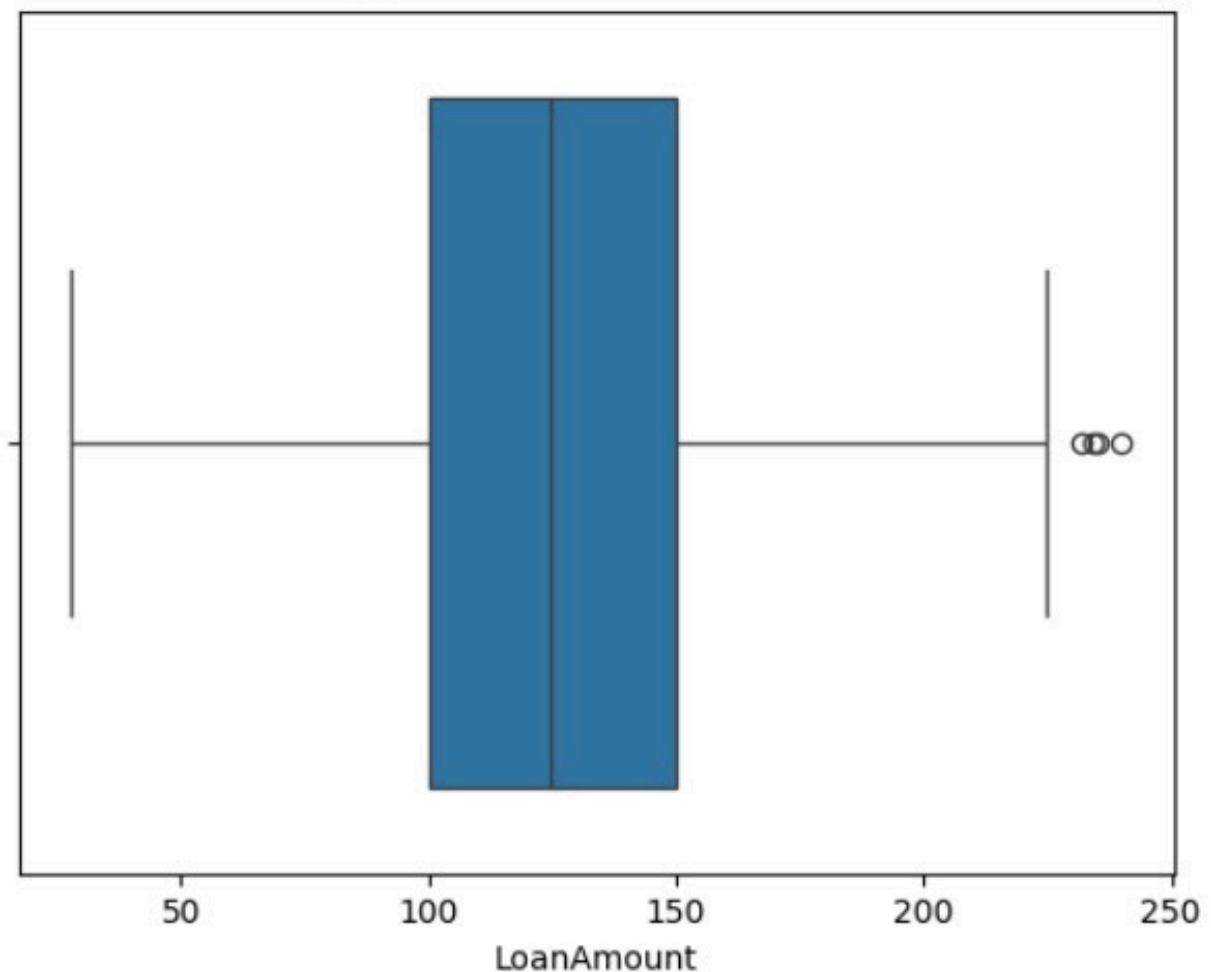
REMOVE OUTLIERS

```
df_clean = df[(col >= lower_fence) & (col <= upper_fence)]
```

Verify After Cleaning

```
import seaborn as sns  
sns.boxplot(x=df_clean['LoanAmount'])
```

... <Axes: xlabel='LoanAmount'>



```
df['LoanAmount'].mean()
```

... np.float64(136.13259668508286)

```
df['LoanAmount'].fillna('136.13259668508286', inplace=True)
```

```
df["Loan_Amount_Term"].median()
```

```
... 360.0
```

```
df["Loan_Amount_Term"].fillna(360, inplace=True)
```

```
df["Credit_History"].median()
```

```
... 1.0
```

```
df["Credit_History"].fillna(1.0, inplace=True)
```

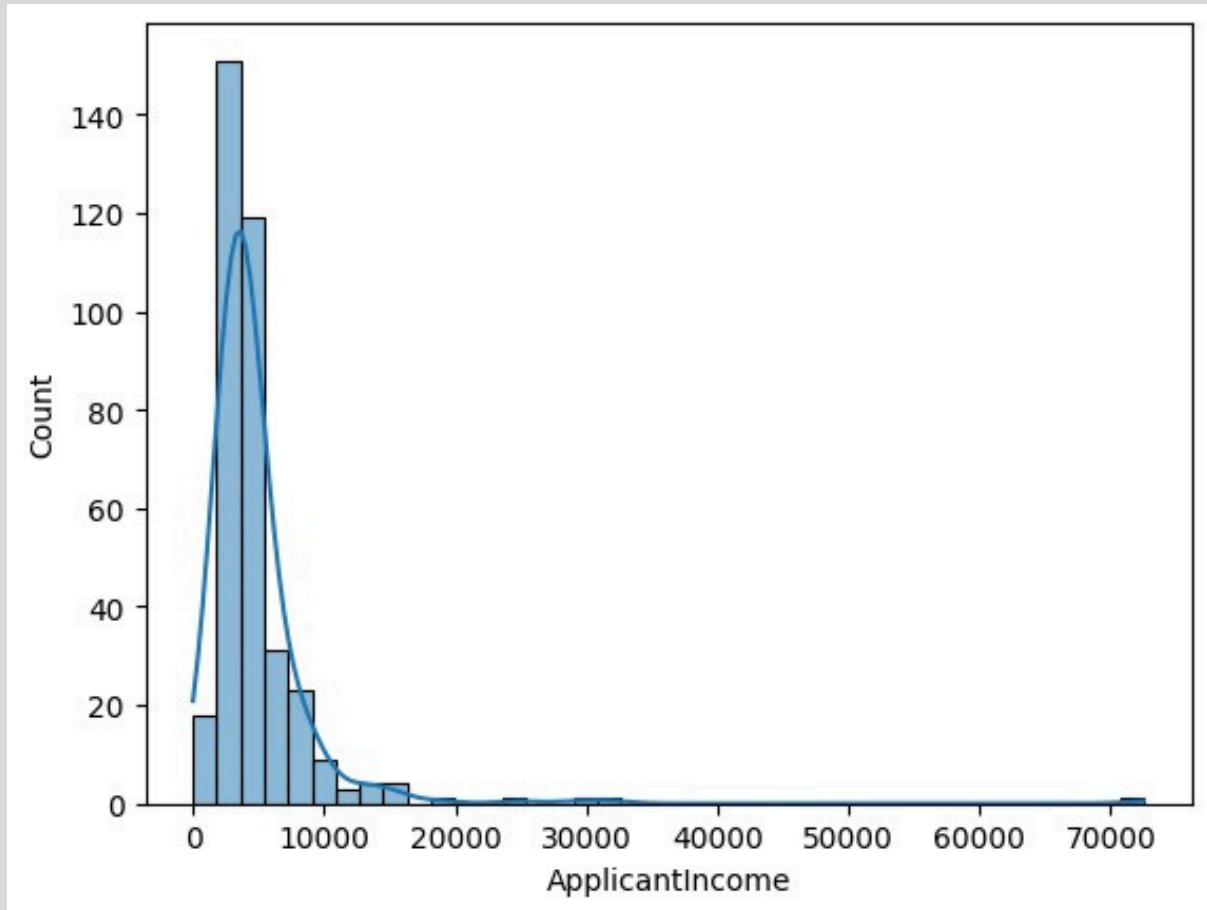
```
df.isnull().sum()
```

**FILL ALL THE NULL
VALUES**

	0
Loan_ID	0
Gender	0
Married	0
Dependents	0
Education	0
Self_Employed	0
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	0
Loan_Amount_Term	0
Credit_History	0
Property_Area	0

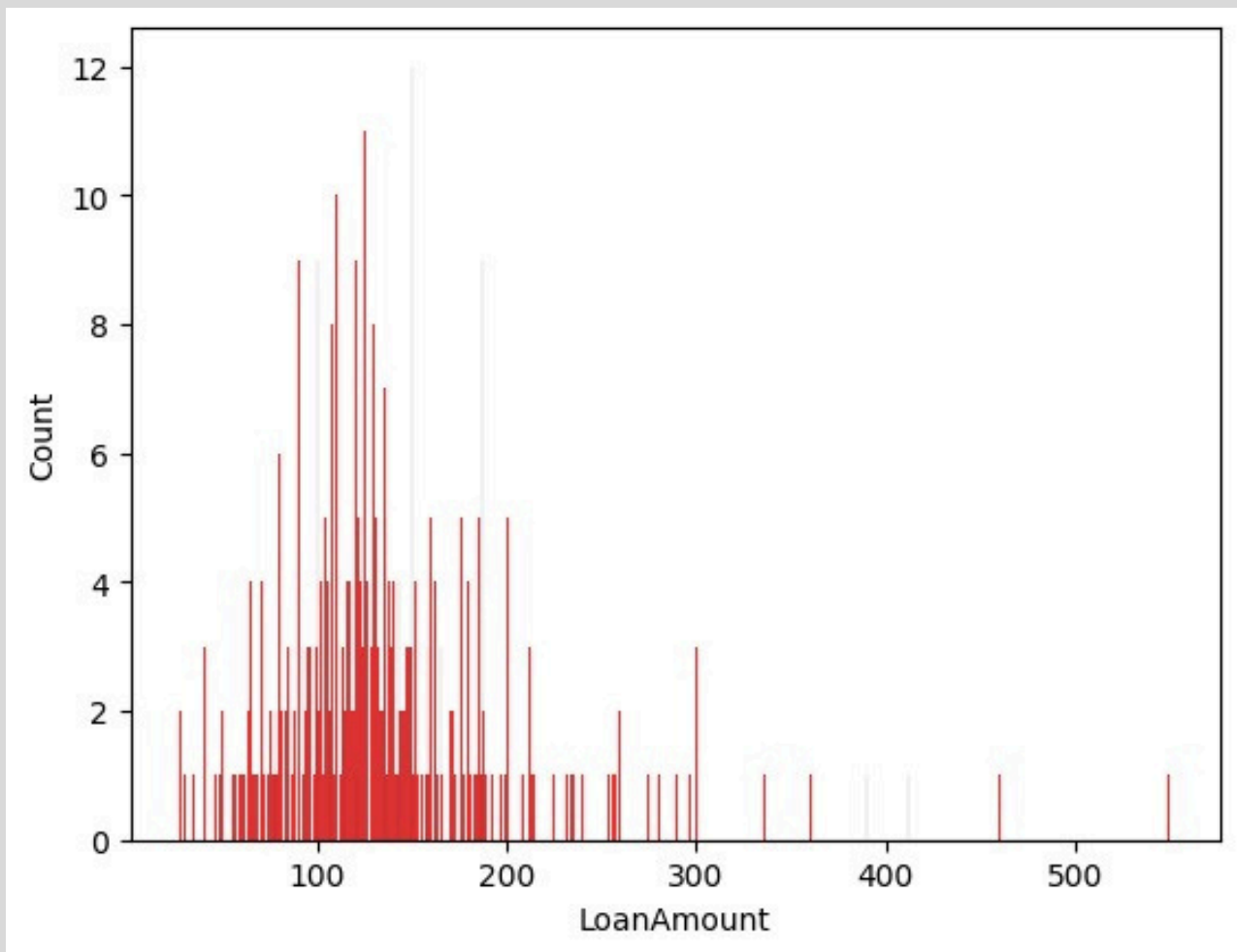
DATA VISUALIZATION

```
import matplotlib.pyplot as plt
import seaborn as sns
histogram=sns.histplot(x="ApplicantIncome",bins=40,kde=True,data=df,legend=False)
```



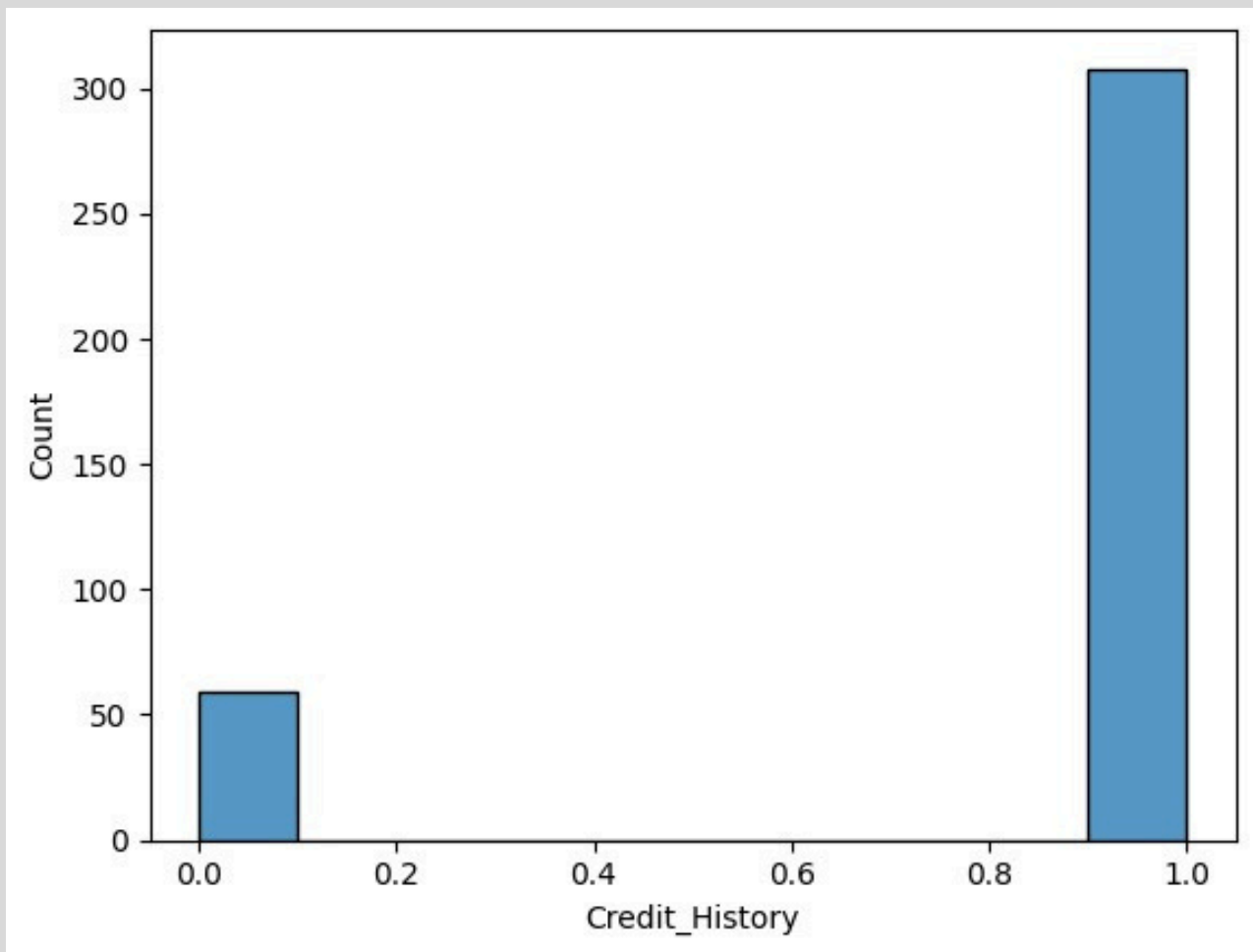
Most applicants earn on the lower side, mainly between about 2,000 and 8,000. As income gets higher, there are fewer and fewer people. A small number of applicants earn much more than the rest, which stretches the graph to the right. Overall, most people have moderate incomes, with only a few high earners.

```
histogram=sns.histplot(x="LoanAmount",data=df,legend=False,color="Red")
```



Most loan amounts are in a middle range, mainly around 80 to 200. As the loan amount increases, fewer people take those higher loans. There are a few very large loan amounts, but they are rare. Overall, most applicants choose moderate loan amounts, with only a small number going for very high loans.

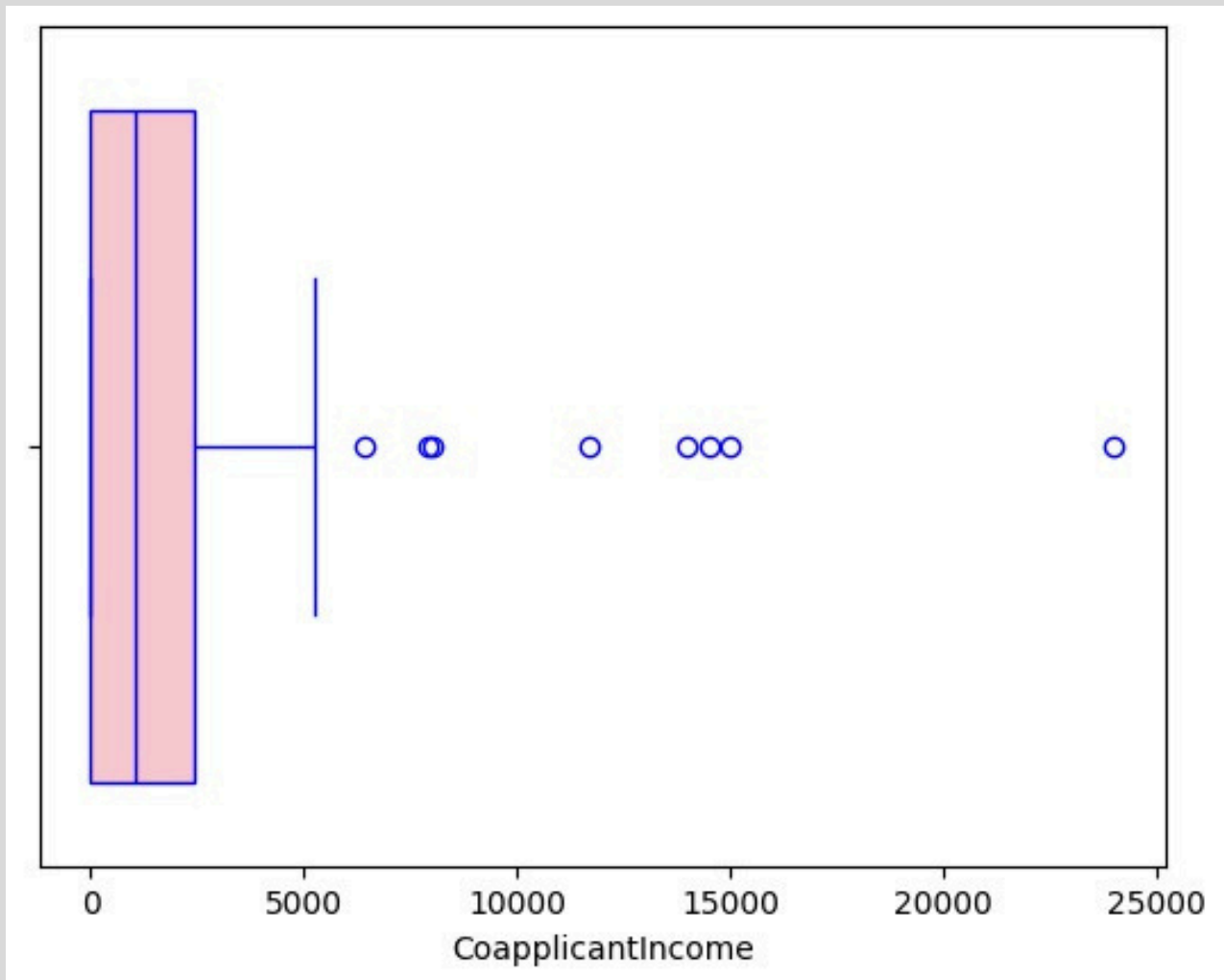
```
histogram=sns.histplot(x="Credit_History",data=df,legend=False)
```



Most applicants have a good credit history (shown by 1), while only a smaller number have a poor or no credit history (shown by 0). This means the majority of people in the data are more likely to be reliable in repaying loans.

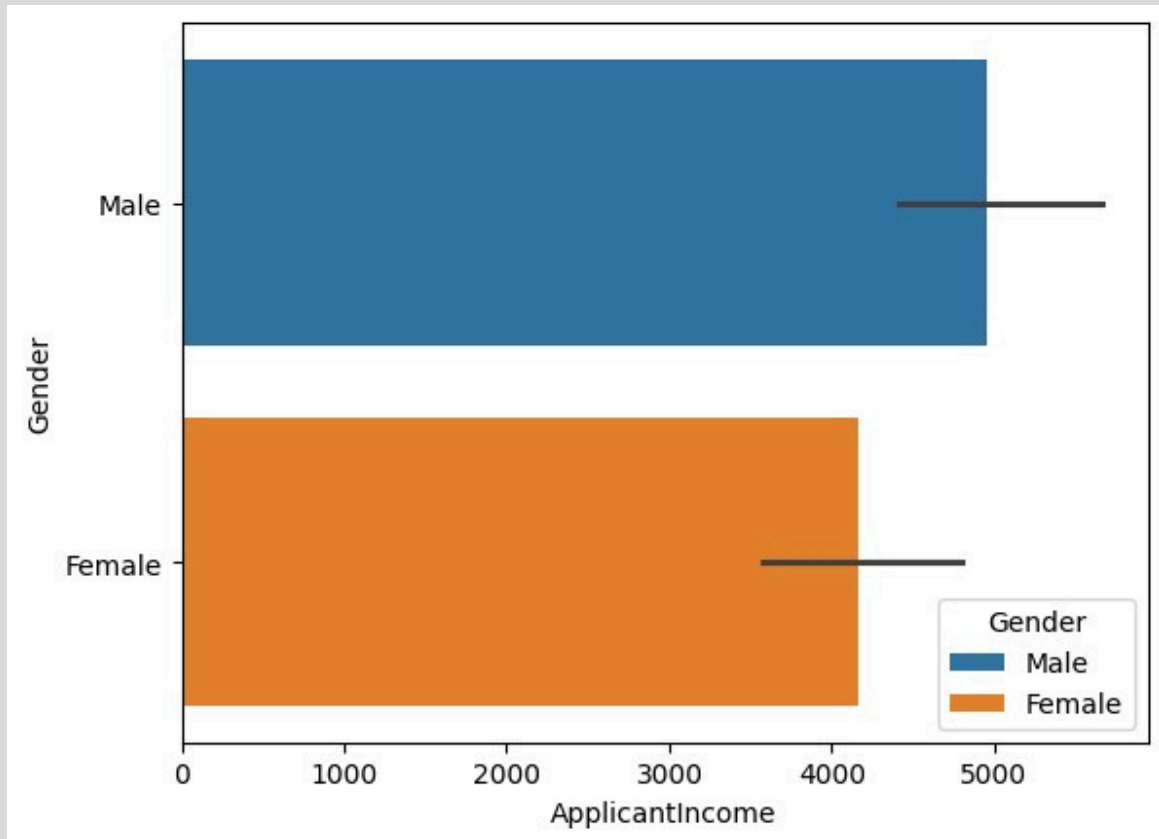
USING BOX PLOTS TRY TO IDENTIFY POTENTIAL OUTLIERS AND VISUALIZE THE SPREAD OF DATA.

```
BoxPlot=sns.boxplot(x="CoapplicantIncome",data=df,color="pink",linecolor="blue")
```



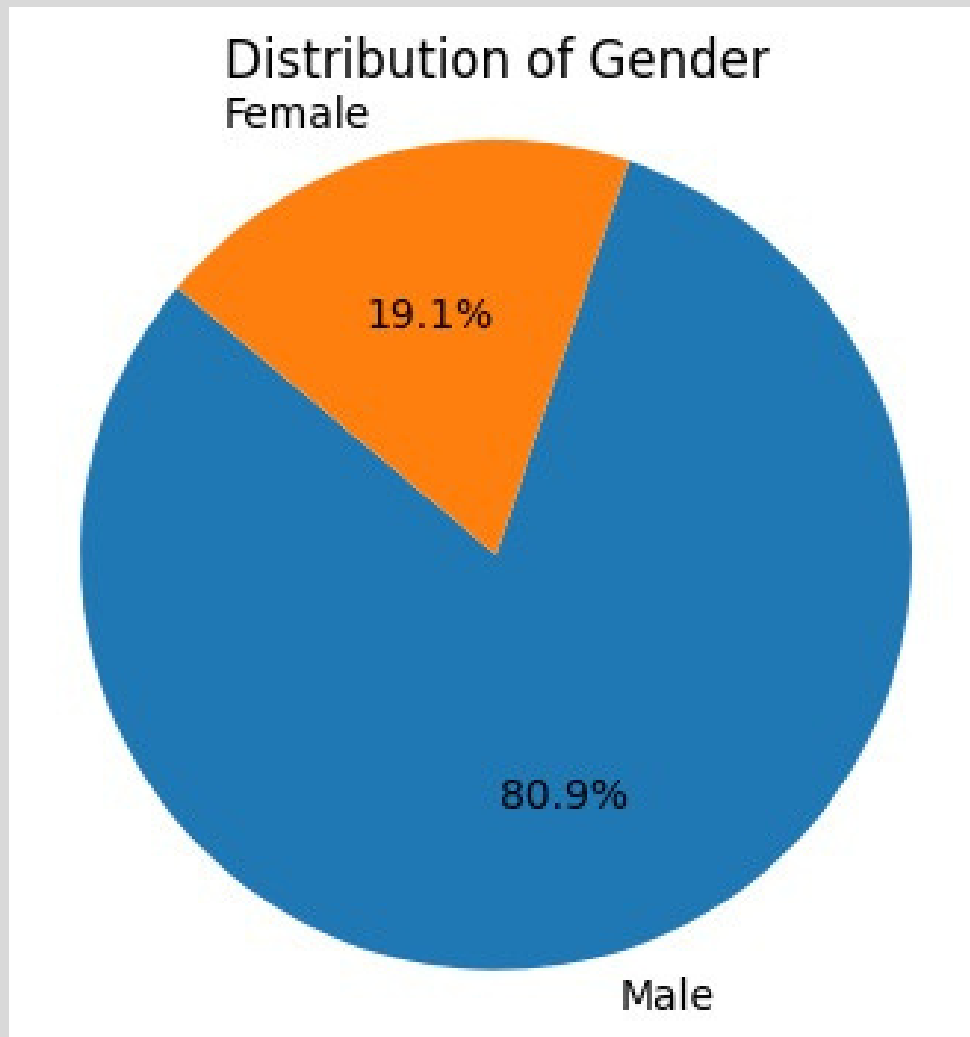
ANALYZE CATEGORICAL VARIABLES

```
import seaborn as sns
import matplotlib.pyplot as plt
plot=sns.barplot(x="ApplicantIncome",y="Gender",data=df,
                 legend=True,hue="Gender")
```



The chart shows that male applicants have a higher average income than female applicants. While there is some variation in both groups, men generally earn more on average compared to women in this data.

```
# Calculate the counts of each category in the 'Gender' column
gender_counts=df['Gender'].value_counts()
# Create a pie chart for the 'Gender' column
plt.figure(figsize=(4, 4))
plt.pie(gender_counts,labels=gender_counts.index,autopct='%1.1f%%',startangle=140)
plt.title('Distribution of Gender')
plt.axis('equal')
plt.show()
```

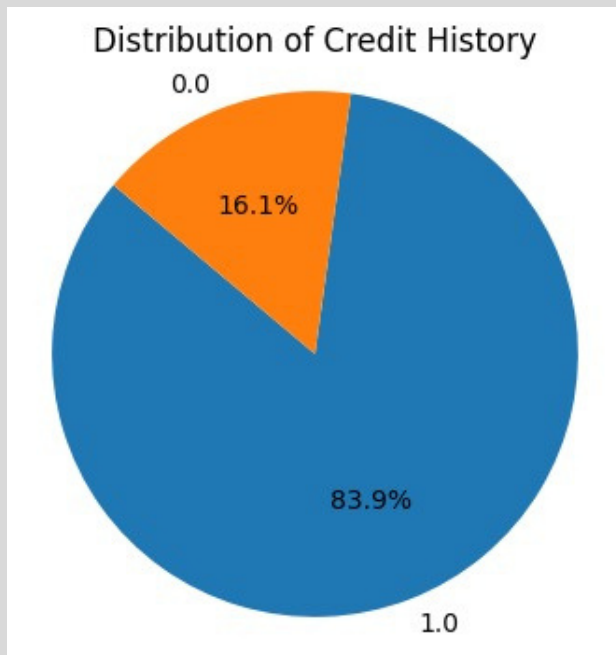


The chart shows that most applicants are male, making up about 81% of the total, while females account for only around 19%. This means the data is not balanced, with far more male applicants than female applicants.

```

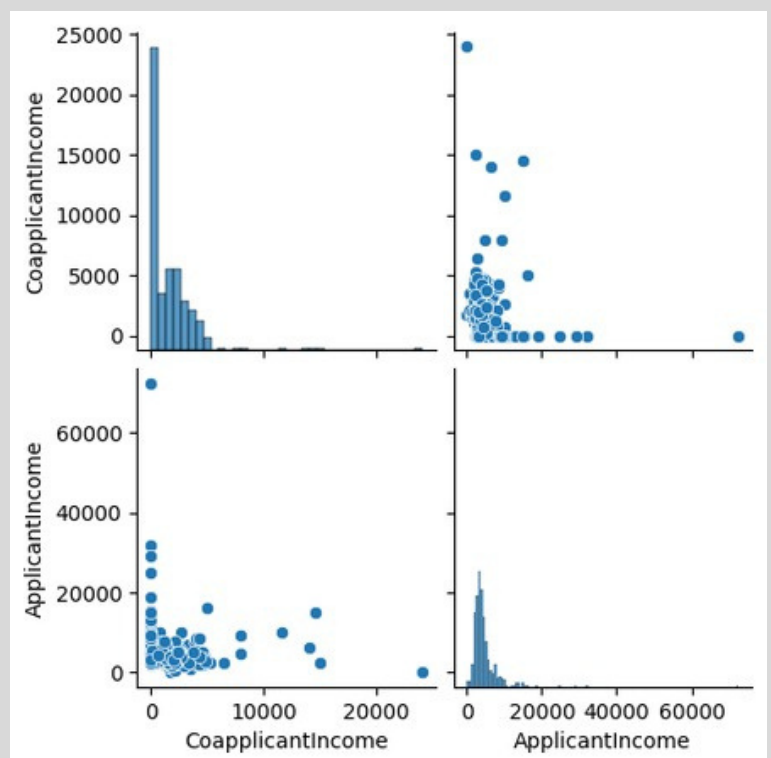
import matplotlib.pyplot as plt
credit_history_counts=df['Credit_History'].value_counts()
plt.figure(figsize=(4, 4))
b = plt.pie(credit_history_counts, labels=credit_history_counts.index,
            autopct='%1.1f%%', startangle=140)
plt.title('Distribution of Credit History')
plt.axis('equal')
plt.show()
sns.pairplot(df[["CoapplicantIncome", "ApplicantIncome"]], markers=4)

```



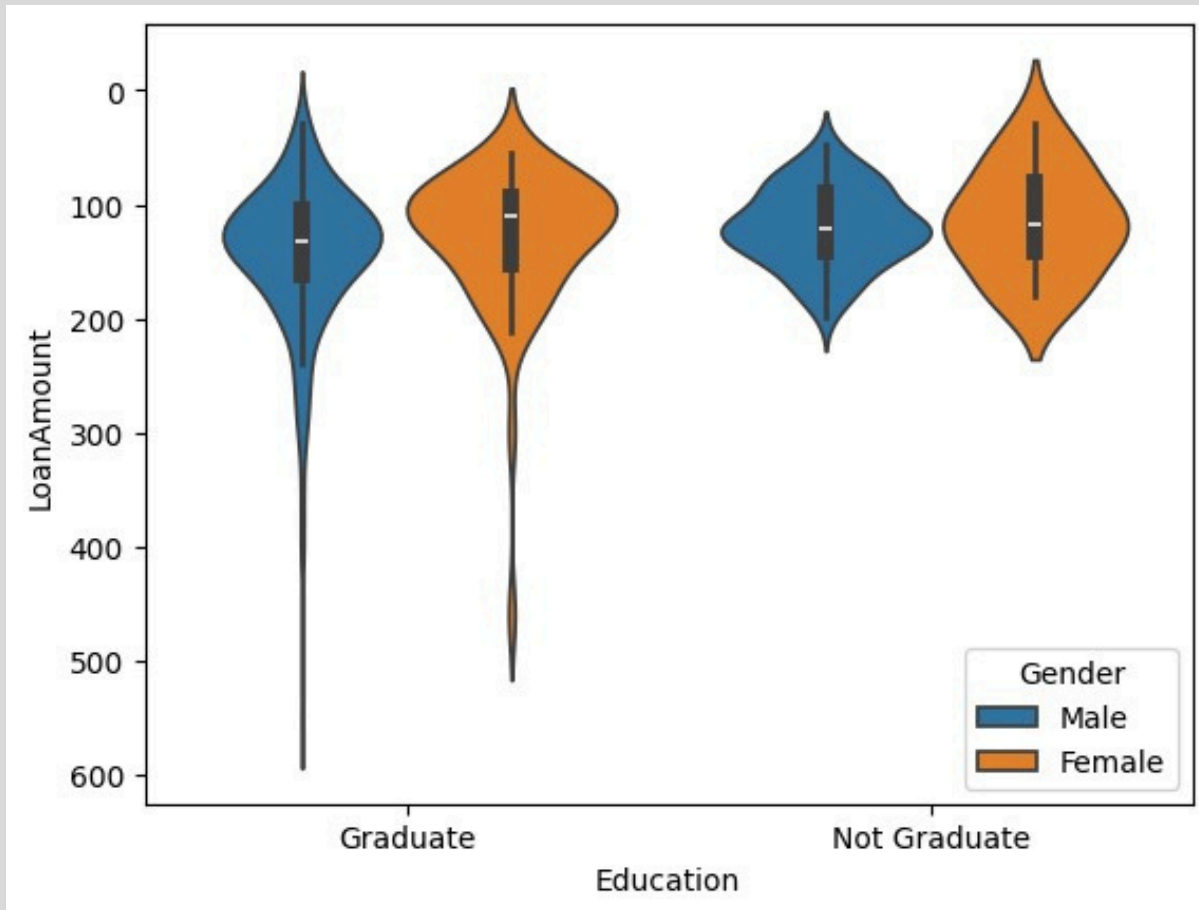
Most people in the dataset have a good credit history. This means having a good credit record is common and may play an important role in loan approval.

Overall, income levels are mostly concentrated at the lower range, and having a higher income in one does not necessarily mean a higher income in the other.



BIVARIATE ANALYSIS

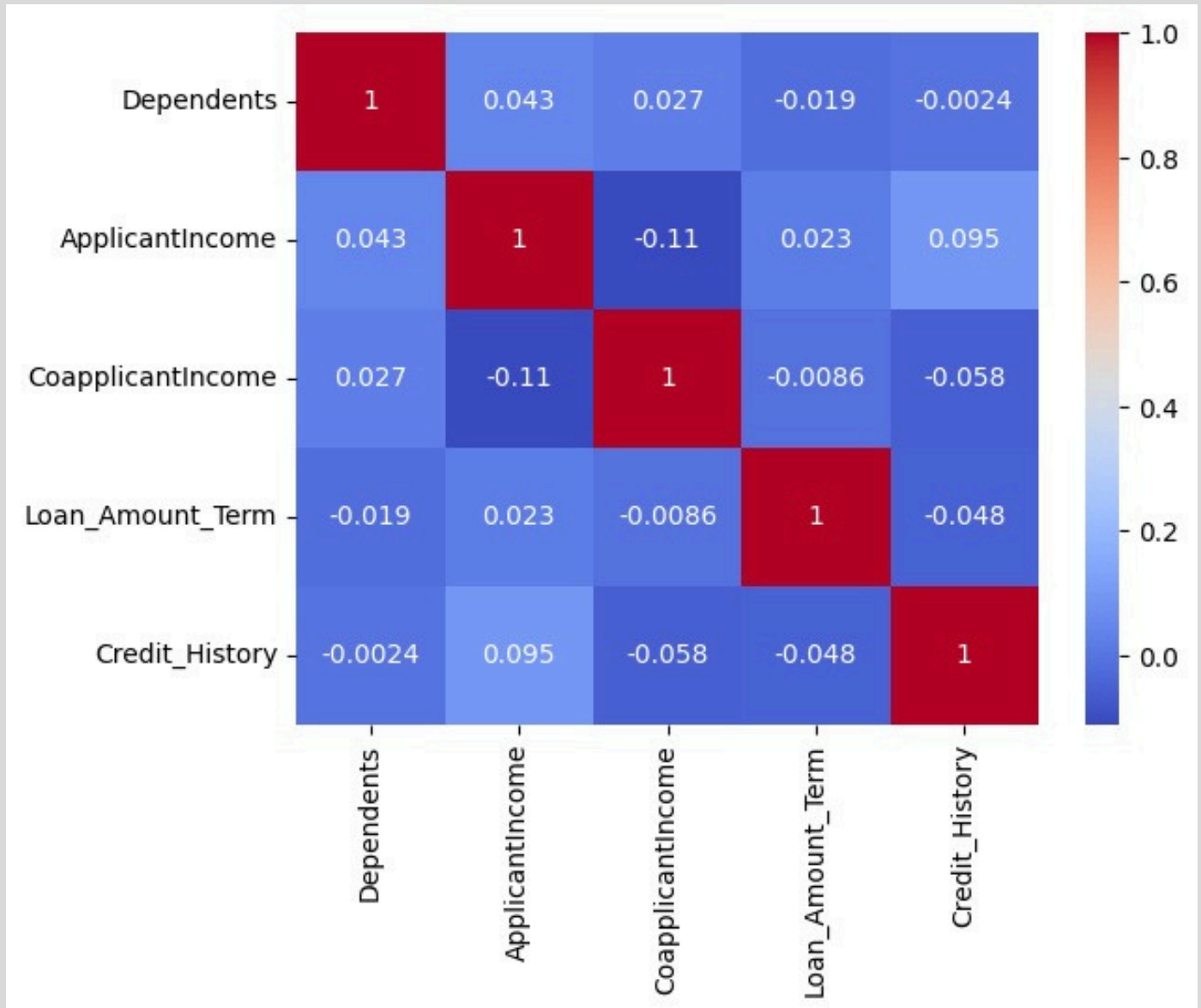
```
sns.violinplot(x="Education",y="LoanAmount",data=df,hue="Gender")
```



Loan amount is quite similar across both education levels and genders. Neither education nor gender seems to have a strong impact on how much loan is taken.

MULTIVARIATE ANALYSIS

```
numerical_df = df.select_dtypes(include=np.number) #For selection of Numeric only  
sns.heatmap(numerical_df.corr(), annot=True, cmap='coolwarm')
```



Overall, none of the features strongly influence each other. This means each variable mostly acts independently, and no pair has a strong impact on the other.

CONCLUSION

1. The loan approval project showcases the effective use of data analysis to support financial decision-making.
2. It demonstrates how machine learning models can predict loan eligibility with improved accuracy.
3. Key factors such as income, credit history, and employment status play a crucial role in determining approvals.
4. The system helps reduce manual errors and speeds up the loan evaluation process.
5. It ensures greater consistency and fairness compared to traditional approval methods.
6. Proper data preprocessing and feature selection are essential for achieving reliable results.
7. Model evaluation techniques help in selecting the most suitable algorithm for prediction.
8. The project highlights opportunities for further enhancement through additional data and tuning.
9. It provides valuable insights for financial institutions to minimize risk and improve efficiency.
10. Overall, the project contributes toward building smarter, automated loan approval systems.